GPU Programming compact cheatsheet

STEP 1 Why GPUs

1.1 Why GPUs: latency, throughput and power

Two designs CPU: big control and caches, few strong ALUs, low latency. GPU: thousands of small ALUs, small control, high throughput.

Latency and throughput Latency = time for one job. Throughput = jobs ÷ time. FPS = 1000 ÷ ms per frame. Batch time = ⌈N ÷ C⌉ × time per round.

Power P ∝ α C V² f. Higher clock needs higher voltage, so power explodes. GPUs use many cores at a moderate clock.

Peak Peak = FLOPs per tick per core × clock × cores. FMA = 2 FLOPs. T4 ≈ 8.1, A100 FP32 ≈ 19.5 TFLOP/s.

Hardware spectrum CPU → GPU → FPGA → ASIC: less flexible, more efficient. TPU and NPU are ASIC style AI accelerators. The CPU is always the host.

Latency and throughput
Latency = time to finish ONE job
Throughput = jobs finished ÷ time taken

Time for a batch
$T = \lceil N \div C \rceil \times$ (time per round)

Dynamic power $P_{\text{dyn}} \propto \alpha \cdot C \cdot V^2 \cdot f$

Peak work per second
peak = (FLOPs per tick per core) × f × cores

TRAP GPU does not mean "always faster". A GPU wins only when there is enough independent work to keep thousands of small units busy. For a single small task the CPU usually wins.

EXAM A GPU never runs alone. The CPU boots the machine, runs the serial parts and launches GPU work. This is why the time to copy data between them matters so much (chapter 1.2).

1.2 Speedup, the PCIe trip and Amdahl's law

Speedup S = old ÷ new. Count copy in + kernel + copy out. Copy time = bytes ÷ bandwidth (PCIe about 12 to 16 GB/s).

Amdahl S(N) = 1 ÷ ((1 − P) + P/N). Ceiling 1 ÷ (1 − P). Efficiency S ÷ N. Back: P = (1 − 1/S) ÷ (1 − 1/N).

Part speedup Speeding a part p by s gives 1 ÷ ((1 − p) + p/s). Speed up the biggest part first.

Gustafson Scaled speedup N − (1 − P)(N − 1) when the problem grows with N (weak scaling).

Keep data on the GPU Copy once, run many kernels, copy back once. Small jobs have a break even size below which the CPU wins.

Speedup S = old time ÷ new time

Total GPU time
$T_{\text{GPU}} = T_{\text{copy in}} + T_{\text{kernel}} + T_{\text{copy out}}$

Amdahl's law $S(N) = 1 \div [\,(1 - P) + P \div N\,]$

The ceiling $S_{\max} = 1 \div (1 - P)$

Gustafson's law (scaled speedup)
$S_{\text{scaled}} = N - (1 - P)(N - 1)$

TRAP Keep data on the GPU across many kernel launches. Copy in once, run all time steps, copy out once. Copying every step can turn a 158 times speedup into 12 times (workbook P1.3.5).

TRAP P is a fraction of the time on one processor, not a fraction of the lines of code. A tiny loop that runs for 90 percent of the time gives P = 0.9.

EXAM In an exam, Amdahl is the default unless the question says the problem grows with the number of processors.

1.3 First GPU workloads: heat simulation and convolution

Heat rule T_new = T + α(up + down + left + right − 4T) + βQ. 1-D uses −2T. Updates = cells × steps.

Why parallel Each cell needs only old neighbour values, so one thread per cell. Keep two buffers (old and new).

Stability Explicit scheme: α ≤ 1/2 in 1-D, 1/4 in 2-D, 1/6 in 3-D, or values blow up.

Convolution cost Normal H W Cin Cout K². Separable H W Cin K² + H W Cin Cout. Ratio (K² + Cout) ÷ (K² Cout) → 1/K².

Same pattern Heat updates and convolutions are stencils: each output reads a small neighbourhood. Tiling with halos speeds them up later.

2-D update rule (5 points)
$T^{\text{new}} = T + \alpha\,(\text{up} + \text{down} + \text{left} + \text{right} - 4T) + \beta Q$

Normal convolution
mults = $H \times W \times C_{\text{in}} \times C_{\text{out}} \times K \times K$

Saving ratio
separable ÷ normal = $(K^2 + C_{\text{out}}) \div (K^2 \times C_{\text{out}})$

TRAP A GPU heat code must keep two buffers. If threads wrote into the same grid they read from, some would read a neighbour's new value and some its old value, and the answer would depend on timing.

STEP 2 The CUDA programming model

2.1 Your first CUDA program

Host and device Host = CPU + RAM, device = GPU + its DRAM. Separate memories, so copy with cudaMemcpy (or use unified memory).

Recipe cudaMalloc, cudaMemcpy H2D, kernel<<<blocks, threads>>>, cudaMemcpy D2H, cudaFree. blocks = (n + B − 1) / B.

Qualifiers __global__ void kernel (host launches), __device__ helper (device only), __host__ default. Kernels cannot return values.

Guard if (i < n) because the last block can overhang the array.

Asynchronous Launch returns at once. cudaMemcpy and cudaDeviceSynchronize wait. Check errors with cudaGetLastError after a launch.

EXAM Quiz 1 question 6: the correct kernel signature is __global__ void. A kernel cannot return a value, because thousands of threads run it and the launch returns to the host at once without waiting. Results go back through memory. __shared__ is a memory space, not a function qualifier.

TRAP The order of the six lines is not guaranteed. Blocks may run in any order, so block 1 can print first. Never write code that depends on the order blocks run in.

2.2 Threads, blocks and grids: indexing in 1-D, 2-D and 3-D

Global index i = blockIdx.x × blockDim.x + threadIdx.x. Blocks = (N + B − 1)/B. Guard i < N. Idle = blocks × B − N.

2-D col = bx·Bx + tx, row = by·By + ty, index = row × width + col. Grid = (⌈W/Bx⌉, ⌈H/By⌉), guard both.

Linear order thread = tx + ty·Bx + tz·Bx·By (x fastest). Warps are cut from this order.

Backwards block = ⌊g ÷ per block⌋, leftover = g mod per block, x = leftover mod Bx, y = ⌊leftover ÷ Bx⌋.

Grid stride Start at i = idx, keep going while i < n, and add blockDim.x × gridDim.x each time. A fixed grid then covers any N.

Global index
i = blockIdx.x × blockDim.x + threadIdx.x

Blocks needed
blocks = $\lceil N \div B \rceil = (N + B - 1) \div B$

Pixels in an image
col = blockIdx.x × blockDim.x + threadIdx.x
row = blockIdx.y × blockDim.y + threadIdx.y
memory position = row × width + col

Position to one number (x fastest)
thread = $t_x + t_y B_x + t_z B_x B_y$
block = $b_x + b_y G_x + b_z G_x G_y$
global = block × (threads per block) + thread

EXAM Exams love going backwards. Always divide by the number of threads per block first, then split the leftover with mod and divide, x first.

TRAP In matVecMul neighbouring threads read A[i × N + j] for the same j, which are N floats apart. That is a strided, uncoalesced read (chapter 3.2). A transposed layout or one warp per row is faster for large N.

2.3 SMs, warps and hiding latency

Mapping Thread → core lane, warp → scheduler, block → one SM for life, grid → whole GPU.

Warps warps = ⌈B/32⌉, warp = ⌊t/32⌋, lane = t mod 32, t = x + y·Bx. Efficiency B ÷ (32 × warps).

Little's law In flight = latency × throughput. Warps = that ÷ 32. Bytes in flight = bandwidth × latency.

Latency hiding Zero cost warp switching because registers stay on chip. Need many warps or independent instructions.

Registers Registers per thread = file ÷ threads. 65,536 on modern SMs. Too many spill to slow local memory.

Warps per block $\text{warps} = \lceil B \div 32 \rceil$

Warp and lane of thread t
$\text{warp} = \lfloor t \div 32 \rfloor$, $\text{lane} = t - 32 \times \text{warp}$

Little's law
jobs in flight = latency (ticks) × throughput (jobs per tick)

TRAP In class it was said that grids sit inside cores. It is the other way round: a thread runs on a core, a block on one SM, and a grid across all SMs.

2.4 Occupancy: how many warps fit on an SM

Occupancy Active warps ÷ max warps per SM. Blocks = min(warp limit, block limit, register limit, shared memory limit).

Block limit Small blocks hit the block cap. Smallest block for 100 percent = max threads ÷ max blocks (A100: 64).

Registers Per warp = ⌈32r ÷ 256⌉ × 256. Blocks = ⌊⌊65,536 ÷ per warp⌋ ÷ warps per block⌋.

Shared memory Blocks = ⌊pool ÷ (bytes + 1 KB reserved)⌋ on CC 8.x.

Not the goal Enough warps to hide latency is enough. Measure achieved occupancy in Nsight Compute.

Occupancy
occupancy = active warps per SM ÷ maximum warps per SM

Blocks that fit
blocks = min(warp limit, block limit, register limit, shared memory limit)

Registers per warp
per warp = ⌈(r × 32) ÷ 256⌉ × 256

Register limit in blocks
blocks = ⌊⌊65,536 ÷ per warp⌋ ÷ warps per block⌋

Shared memory limit
blocks = ⌊shared memory per SM ÷ (bytes per block + reserved)⌋

STEP 3 GPU memory

3.1 The GPU memory hierarchy

Spaces Register (thread), local (thread, in DRAM), shared (block), global (all, host, until cudaFree), constant (all, read only).

Q8 Visible to all SMs for the whole application: global memory. L2 is a cache, not a space.

Constant 64 KB, host writes with cudaMemcpyToSymbol, broadcast when a warp reads one address.

Host memory Pageable malloc, pinned cudaMallocHost (needed for async overlap), managed cudaMallocManaged with prefetch.

Copies Copy time = bytes ÷ bandwidth. PCIe is far slower than device DRAM. Copy once, reuse many times.

Copy time t = bytes ÷ bandwidth

TRAP Local memory is not near the thread. "Local" means private. It lives in device DRAM and is only cached on chip.

EXAM A memory space is something you name and address. A cache is a hardware copy of global data that fills and empties by itself. This difference decides questions like Q8.

3.2 Coalescing: reading global memory well

Sectors Warp loads are coalesced into 32 byte sectors. Efficiency = 32 × element ÷ (sectors × 32).

Strides Floats: stride 1 → 4 sectors, 2 → 8, 4 → 16, ≥ 8 → 32. Misaligned start adds a sector.

Row major Let threadIdx.x move along the row. Q5 store B[x·h + y] touches 32 segments.

Sector model Useful bandwidth ≈ stride 1 bandwidth ÷ min(s, 8). T4 206 GB/s at stride 1.

Fixes SoA instead of AoS, float4 loads, and shared memory tiles for transposes.

Bytes the warp wants
useful = 32 × element size

Efficiency
efficiency = useful bytes ÷ (sectors × 32)

Sector model
useful bandwidth ≈ (stride 1 bandwidth) ÷ min(s, 8)

3.3 Shared memory and bank conflicts

Shared memory __shared__, one copy per block, fill then __syncthreads() then read. extern __shared__ size is the third launch value.

Banks 32 banks × 4 bytes. bank = ⌊byte ÷ 4⌋ mod 32. Different words in one bank take turns.

Conflicts Stride s gives gcd(s, 32) ways. Same word is a broadcast, no conflict.

Padding Tile [32][33] makes column reads conflict free for 128 extra bytes.

Layout Let lanes write consecutive words: powers example 1024 passes against 32.

Bank of an address
bank = ⌊byte address ÷ 4⌋ mod 32

Conflict for stride s ways = gcd(s, 32)

TRAP __syncthreads() must be reached by every thread of the block. Putting it inside an if that only some threads enter can hang the block or give wrong results.

STEP 4 Numbers and performance limits

4.1 Floating point numbers on the GPU

Format value = (−1)^S × 1.f × 2^(E − bias). Float 1/8/23 bias 127, double 1/11/52 bias 1023.

Exact Exact in binary only when the denominator is a power of 2. 5.6 is stored as 5.599999904632568.

Add Align exponents, add mantissas, normalise, round. Not associative: compare with a tolerance.

FMA a × b + c with one rounding, one instruction, counted as 2 FLOPs.

Formats FP16 1/5/10 (max 65,504), BF16 1/8/7, TF32 1/8/10. Epsilon = 2^−mantissa bits.

IEEE-754 value $\text{value} = (-1)^S \times 1.f \times 2^{E - \text{bias}}$

TRAP The lecture 5 notes give 1.00101_2 = 1.5625. Adding the bits gives 1 + 0.125 + 0.03125 = 1.15625 = 37/32, which is the value the workbook uses.

EXAM A GPU reduction adds numbers in a different order from a CPU loop, so float sums can differ in the last digits. That is expected. Compare float results with a tolerance, never with ==.

4.2 FLOPs, bandwidth and the roofline

FLOPs Matmul 2n³, k × k filter 2k² per output. Peak = SMs × cores × 2 × clock. FMA = 2 FLOPs.

Bandwidth GB/s = (read + written bytes) ÷ 10^9 ÷ s. Count both directions.

AI FLOPs ÷ bytes. SAXPY 1/6, plain matmul 0.25, perfect reuse n/8 float, n/16 double.

Roofline Attainable = min(peak, AI × B). Ridge = peak ÷ B. Left memory bound, right compute bound.

Time bound t ≥ max(FLOPs ÷ peak, bytes ÷ bandwidth). Memory bound kernels gain nothing from more cores.

Matrix multiply n × n FLOPs = $2n^3$

Peak of a GPU
peak = SMs × cores per SM × 2 (FMA) × clock

Bandwidth
GB/s = (bytes read + bytes written) ÷ 10^9 ÷ seconds

Arithmetic intensity
AI = FLOPs ÷ bytes moved to or from DRAM

Attainable speed
attainable = min(peak FLOP/s, AI × bandwidth)

Ridge point $\text{AI}_{\text{ridge}} = \text{peak} \div \text{bandwidth}$

EXAM For a memory bound kernel, doubling the maths units changes nothing. Only more bandwidth or more reuse helps. Doubling Fermi's cores gives plain matrix multiply no gain.

STEP 5 Parallel patterns

5.1 Parallel reduction

Tree $\log_2$ n levels, n − 1 additions, 1 + levels barriers. 1024 → 10 levels, 11 barriers.

Pairing Sequential (t < s, s halves) packs active threads: only levels with s < 32 split a warp.

Barrier rule Every thread must reach every __syncthreads(). Never put it inside a divergent if.

Grid One atomicAdd per block (⌈n/B⌉ atomics) or repeated launches until one value is left.

Shuffles __shfl_down_sync with offsets 16 to 1 sums a warp in 5 steps without shared memory.

Levels $\text{levels} = \lceil \log_2 n \rceil$

Additions total additions = n − 1

Atomics needed one per block = ⌈n ÷ B⌉

5.2 Tiled matrix multiplication

Plain 2N loads per thread, 2N³ total (not N² as slide 8 says). Q2: 524,288 per block.

Tiled ⌈N/T⌉ phases, 2⌈N/T⌉ loads per thread, 2N³/T total. Q3: 32,768 per block.

Barriers Sync after loading the tile and again before the next phase. Q4: without them both a and b happen (answer 3).

Ragged Pad out of range loads with 0 and guard the store. n = 5, T = 4: 4 blocks, 39 idle.
Limits $T^2 \le 1024$ threads so $T \le 32$. Shared memory $8T^2$ bytes. AI $\approx T/4$.

Loads per thread (plain) $2N$

Loads per block (plain)
threads per block $\times 2N$

Phases phases $= \lceil N \div T \rceil$

Loads per thread (tiled) $2 \times \lceil N \div T \rceil$

Total loads (tiled)
$2N^2\lceil N \div T \rceil = 2N^3 \div T$ when T divides N

TRAP Lecture 7 slide 8 states the plain kernel reads Width² values. Each of the Width² threads reads 2 × Width values, so the total is 2 × Width³.

5.3 Stencils, halos and heat simulation

Stencil Each output from a fixed neighbour pattern. 2-D: ±1, ±W. 3-D: ±1, ±N, ±N².
Halo Tile B + 2R. Reads per output $(B + 2R)/B$ against 2R + 1. Barrier after loading.
3-D tiles $AI = 13(T-2)^3/(4T^3) \le 3.25$. Halo share 57.8 percent at T = 8. Memory bound.
Heat Tnew = $(1 - 6\lambda)T + \lambda\Sigma 6$. Weights must add to 1 (slide 14 used 0.25 + 0.6). Stable for $\lambda \le 1/6$.
Ping-pong Read the old grid, write the new grid, swap pointers.

Neighbours in 2-D
$N = i - W$, $S = i + W$, $W = i - 1$, $E = i + 1$

Neighbours in 3-D $\pm 1, \pm N, \pm N^2$

Tile size $B + 2R$ values

Global reads per output
$(B + 2R) \div B$ with the tile, $2R + 1$ without

With T × T × T tiles
$AI(T) = 13 \times (T-2)^3 \div (4T^3)$

Halo share $(T^3 - (T-2)^3) \div T^3$

3-D explicit heat step
$T_{new} = (1 - 6\lambda)\,T + \lambda \times$ (sum of 6 neighbours)

Stability
$\lambda \le 1/6$ in 3-D, $1/4$ in 2-D, $1/2$ in 1-D

TRAP Lecture 6 slide 14 uses 0.25 for the centre and 0.1 for each of the 6 neighbours. They add to 0.85, so a field of 100 drops to 85 every step. With neighbour weight 0.1 the centre must be $1 - 6 \times 0.1 = 0.4$.

5.4 Convolution and Sobel edge detection

Size out $= \lfloor (W + 2P - K)/S \rfloor + 1$. Same padding $P = (K-1)/2$.
Work MACs = out² K² Cin Cout. FLOPs = 2 × MACs.
Sobel Gx, Gy 3 × 3. M1 = |Sx| + |Sy| clamped to 255. Weights in constant memory.
Tiles Tile $(B + 2r)^2$, loads per output $(B + 2r)^2/B^2$, best saving $1 - 1/K^2$.
Correction Sharpening 30 with 25, 35, 20, 40 gives 30, not 35. Smooth slopes do not change.

Output size out $= \lfloor (W + 2P - K) \div S \rfloor + 1$

Work multiply-adds $= \text{out}^2 \times K^2 \times C_{in} \times C_{out}$

Edge strength
$M1 = |S_x| + |S_y|$, $M2 = \sqrt{(S_x^2 + S_y^2)}$

Tile side $B + 2r$, with $r = (K-1) \div 2$

Loads per output $(B + 2r)^2 \div B^2$

TRAP Lecture 8 sharpens the pixel 30 at (2, 1) to 35. With neighbours 25, 35, 20 and 40 the result is 5 × 30 − 120 = 30. On a smooth slope sharpening changes nothing.

5.5 Scan, compaction and histograms

Scan Inclusive y_i = x_0 + ... + x_i. Exclusive starts at 0 and stops before x_i.
Algorithms Hillis and Steele: $\log_2 n$ steps, n $\log_2 n - (n-1)$ adds. Blelloch: 2 $\log_2 n$ steps, $2(n-1)$ adds.
Race Never scan in place in one step. Double buffer or barrier between read and write.
Compaction Exclusive scan of keep flags gives each kept value its output slot.
Histogram Atomics per bin. Privatise in shared memory, merge with at most blocks × bins global atomics.

Inclusive scan $y_i = x_0 + x_1 + \ldots + x_i$

Exclusive scan $y_i = x_0 + \ldots + x_{i-1}$, $y_0 = 0$

Privatised (shared memory) histogram
global atomics = blocks × bins at most

TRAP Updating one array in place with buf[t] += buf[t − off] is a race: thread t may read buf[t − off] after thread t − off has already added to it. Double buffering or a barrier between the read and the write fixes it.

STEP 6 Execution and overlap

6.1 Warp divergence and warp scheduling

Divergence Inside a warp only. Time = sum of used path lengths. Efficiency = lane ticks ÷ (32 × time).
Q9 tid % 4 == 1 → lanes 1, 5, ..., 29, sum 120.
Avoid it Branch on warp number, use selects, keep data dependent branches short.
Schedulers RR waits together, GTO spreads loads (best = (instructions − 1) + L), two level in between.
Hiding Warps = $\lceil (W + c) \div c \rceil$. More ILP per warp means fewer warps needed.

Warp time
time = sum of the lengths of every path at least one lane takes

Efficiency
efficiency = (sum over lanes of their path length) ÷ (32 × warp time)

Best possible finish
finish ≥ (total instructions − 1) + L

Warps to hide a wait warps $= \lceil (W + c) \div c \rceil$

6.2 Streams, asynchrony and overlap

Async Launches and cudaMemcpyAsync return at once. cudaMemcpy and the synchronise calls block the CPU.
Streams In order inside a stream, overlap across streams. Async copies need pinned memory.
Pipeline Equal stages: (N + 2)t, speedup 3N/(N + 2) < 3. Unequal: fill + N × slowest stage.
Danger Not kernels overwriting each other in one stream, but the CPU reading before the GPU is done.
Events cudaEventRecord marks a point. ElapsedTime for timing, StreamWaitEvent for cross stream order.

Serial $t = N \times (t_{in} + t_{kernel} + t_{out})$

Overlapped, equal stages t
$t = (N + 2) \times t$, speedup $3N \div (N + 2)$

TRAP In class it was said that asynchrony is dangerous because a second kernel could overwrite the first. Work in one stream always runs in order. The real danger is the CPU reading or reusing a host buffer before the GPU has finished with it. Synchronise before reading results.

STEP 7 Synchronisation

7.1 Race conditions and locks

Three steps counter++ is load, add, store. Other threads can come between them.
Counting orders $(n \times s)! \div (s!)^n$. Two threads: 20, three threads: 1680. Only the serial orders are correct.
Three rules Mutual exclusion, progress, bounded waiting. Breaking them gives two inside, deadlock, starvation.
Dekker attempts 1 fails progress (strict turns), 2 fails mutual exclusion (not deadlock), 3 deadlocks.
Peterson flag[me] = 1, turn = other, wait while flag[other] and turn == other. Correct, with fences on real hardware.
N threads Bakery: tickets plus id, first come first served. Filter: N − 1 gates, starvation free but not bounded.

Orders of n threads with s steps each
$(n \times s)! \div (s!)^n$

Three threads, three steps $9! \div (3!)^3 = 1680$

TRAP The unsafe result is not guaranteed to be 1. It depends on how the hardware schedules the threads. A race means the answer is unpredictable, which is worse than always wrong.

EXAM Slide 8 says attempt 2 can deadlock. It cannot: a thread sets its flag only after passing its loop, so nobody waits after the flags are set. The real flaw is that both can pass the check, which breaks mutual exclusion. The class also concluded that the three attempts suffer from the same drawback. They fail in three different ways: attempt 2 checks too early, attempt 3 claims too early, attempt 1 is safe but too rigid.

EXAM In class the conclusion was that the deadlock danger is still looming. That is not correct. With reads and writes that happen in program order, Peterson meets all three rules. Swapping steps 1 and 2 (turn first, then flag) does break mutual exclusion, so the order matters.

EXAM Slide 3 says the Filter lock has bounded waiting. It is starvation free (every waiting thread gets in in the end), but a thread can be overtaken any number of times at a gate, so there is no fixed bound r. The Bakery lock is the one with first come, first served order.

7.2 Barriers, atomics and GPU locks

Barrier against lock __syncthreads(): all threads of one block meet. Lock: one thread at a time, only those who want it wait.

Who shares Registers: nothing. Shared memory: __syncthreads(). Global across blocks: atomics, locks or a new kernel.

atomicCAS lock Spin on atomicCAS(&l, 0, 1) until it returns 0, do the work, call __threadfence(), then atomicExch(&l, 0). One thread per block takes it.

Privatisation Count in shared memory, then one global atomicAdd per block. 10 × 256: 2560 global atomics become 10.

Priority counter Same level runs together, levels get skipped, and non resident blocks can deadlock. grid.sync() needs a cooperative launch.

Compare and swap
old = *p, if old = cmp then *p = v, return old

Largest cooperative grid
blocks ≤ (blocks per SM) × (number of SMs)

TRAP Every thread of the block must reach every __syncthreads(). A barrier inside an if that only some threads enter is undefined behaviour: the kernel may hang, or seem to work and give wrong results. The reduction chapter shows the lecture 12 example.

EXAM The slide's critical_kernel lets every thread call acquire_lock. Before Volta (2017) the 32 lanes of a warp move in lock step. One lane wins the lock, but the warp keeps running the spin loop for the losers and never lets the winner reach release_lock, so the kernel hangs. Newer GPUs avoid the hang, but the lock then makes every thread of the grid wait in a single line, which is very slow. Take the lock with one thread per block, and add a fence before the release.

EXAM In the block priority code of slide 7, the counter must be read as volatile or atomically, or the compiler may move the read out of the loop and spin for ever. CUDA does not promise which blocks are running or in what order they start. That this code usually works is luck.

STEP 8 Tensor cores and fast matmul

8.1 Low precision numbers and quantisation

The formula x = (−1)^s × 1.m × 2^(e − bias), bias = 2^(E−1) − 1, gap after 1.0 = 2^−M.

Range against detail Exponent bits give range, mantissa bits give detail. FP16 overflows at 65,504, BF16 keeps FP32 range with about 2 to 3 digits.

Which format FP32 master weights, BF16 training, FP16 with loss scaling, TF32 for FP32 code on tensor cores, INT8 or FP8 for inference.

Wide accumulator Narrow inputs, FP32 sums. In FP16, 2048 + 1 = 2048.

Quantisation q = round(x ÷ s) + z, $\hat{x} = s(q - z)$. Symmetric INT8: s = R ÷ 127. Error at most s ÷ 2 unless clipped.

Fewer bytes, higher intensity
AI = FLOPs ÷ bytes

Value of a normal number
$x = (-1)^s \times 1.m_1 m_2 \ldots m_M$ (binary) $\times 2^{e - \text{bias}}$

Bias $\text{bias} = 2^{E-1} - 1$

Quantise
$q = \text{clip}(\text{round}(x \div s) + z, q_{min}, q_{max})$

Bring back $\hat{x} = s \times (q - z)$

EXAM Slide 11's bars (TF32, FP16 and BF16 at 4 times, FP8 at 8, FP4 at 16) are labelled illustrative. On an A100, TF32 is 8 times and FP16 16 times the FP32 CUDA core rate. Always use the data sheet of the GPU in front of you.

8.2 Tensor cores, GEMM and WMMA

Math limited test AI = ops ÷ bytes > peak ÷ bandwidth. A100 FP16 ridge ≈ 201.

FC layer AI = N_in N_out B ÷ (N_in B + N_in N_out + N_out B). 4096 layer: 4096B ÷ (4096 + 2B), math limited from batch 223.

GEMM 2MNK FLOPs. Convolution: M = out channels, N = batch × output pixels, K = in channels × filter size (implicit GEMM).

Tensor core peak SMs × TCs × FMAs per clock × 2 × clock. V100 125, A100 312, H100 989 TFLOP/s.

WMMA One warp per 16 × 16 tile, fragments, load_matrix_sync, mma_sync (8192 FLOPs), store_matrix_sync. FP16 in, FP32 sum.

Waves waves = ⌈blocks ÷ (SMs × blocks per SM)⌉, efficiency = n ÷ (W × waves). 217 blocks on 108 SMs: 67 percent.

Math limited when
ops ÷ bytes > $\text{BW}_{\text{math}} \div \text{BW}_{\text{mem}}$

4096 × 4096 layer
AI = 4096 B ÷ (4096 + 2B)

Peak of the tensor cores
peak = SMs × tensor cores per SM × FMAs per clock × 2 × clock

Waves
waves = ⌈blocks ÷ (blocks per SM × SMs)⌉

Wave efficiency
n ÷ (W × ⌈n ÷ W⌉), with W = blocks per wave

EXAM Slide 16 says shared memory is used for the matrix C. In real kernels such as CUTLASS, C stays in registers during the main loop and only passes through shared memory when it is written out at the end.

8.3 How fast matmul kernels are built

Speed of light H100 SXM BF16: 132 × 4 tensor cores × 1024 FLOPs × 1.83 GHz ≈ 989 TFLOP/s. Power throttling lowers the real ceiling.

Naive kernel Coalesced order 3171 GFLOP/s (43 ms), swapped order 243 GFLOP/s (565 ms). 13 times from one index.

Tile intensity AI = (2 ÷ b) B_M B_N ÷ (B_M + B_N). Square is best. FP32 T × T: T ÷ 4. BF16: T ÷ 2.

Hopper TMA copies tiles, swizzle slot = chunk XOR row, wgmma on 4 warps, producer and consumer warp groups.

Ladder 32 (no tensor cores) to 317 (tensor cores + TMA) to 764 TFLOP/s, about 77 percent of peak.

Intensity of a BM × BN tile
$\text{AI} = (2 \div b) \times B_M B_N \div (B_M + B_N)$

Square FP32 tile T × T AI = T ÷ 4

EXAM With the numbers used here (756 TFLOP/s BF16 on the PCIe card, 2.0 TB/s) the ridge is about 378 FLOP/byte. The blog quotes about 410 with slightly different specs. Either way a kernel needs hundreds of FLOPs per DRAM byte, so tiles must also be reused through L2 and shared memory.

STEP 9 Tools and Python on the GPU

9.1 Debugging and profiling CUDA code

Order of tools Check every call. Wrong: compute-sanitizer, printf, cuda-gdb. Slow: nsys, then ncu on the hot kernel.

Sanitizer tools memcheck (bounds), racecheck (shared memory races), initcheck (uninitialised reads), synccheck (bad barriers).

cuda-gdb Build with -g -G. set cuda break_on_launch application, cuda block (1,0,0) thread (31,0,0), p x.

Reading a report Time = cycles ÷ clock. SM active ÷ elapsed shows idle SMs. Both units under 60 percent: latency bound, read the stall reasons.

PTX and SASS PTX is virtual and portable (%ctaid.x, ld.global), SASS is the real code (S2R, LDG, FADD, STG). See them with nvcc -ptx, cuobjdump -sass.

TRAP This program usually runs without any message. The stray writes land in memory the program happens to own, so nothing crashes. That is exactly why out of bounds bugs are dangerous and why the sanitizer comes first.

EXAM Slide 6 says threads 0 to 21 write out of bounds. Thread i writes data[i + 10], so even thread 0 writes data[10], one past the end. All 32 threads write out of bounds, to indices 10 to 41.

9.2 GPU programming from Python

Routes CuPy for NumPy code, Numba for kernels in Python, Triton for block level kernels, PyTorch for deep learning. All compile just in time.

Numba twins @cuda.jit, cuda.grid(1), to_device, device_array_like, add[blocks, threads], copy_to_host, shared.array, syncthreads, atomic.add.

Simulator NUMBA_ENABLE_CUDASIM=1 runs kernels on the CPU for testing and debugging.

CuPy Each operation is a kernel and a full memory pass. Fuse with ElementwiseKernel or cp.fuse. RawKernel for CUDA C.

Triton and PyTorch Triton: program_id, offs, mask, tl.load and tl.store. PyTorch: pin_memory, non_blocking, Stream, synchronize.

STEP 10 Exam practice

10.1 Quiz 1 solved step by step

Q1 Use the statement about the SM: 4 FMA × 2 × 1.5 GHz = 12 GFLOPS, option 2.

Q2 and Q3 Naive: B² × 2N loads per block (524,288). Tiled: B² × 2 × N/B (32,768).

Q4 No __syncthreads: both races possible, answer 3. Fix with one barrier after loading and one after computing.

Q5 and Q10 Column store with stride 2048 floats: 32 segments. Warps = blocks × threads per block ÷ 32 = 32,768.

Q6 to Q9 __global__ void, y[2] = 13, global memory, tids 1 to 29 step 4 sum to 120.

EXAM The other reading, 4 FMA per core, gives 64 × 4 × 2 × 1.5 = 768 GFLOPS per SM, which matches no option. When a question gives two numbers for the same thing, use the one that describes exactly what is asked (here the SM) and check the options.

EXAM Answer 3: both statements are true for the quiz code, because it has no __syncthreads. The fixed version above needs one barrier after loading and one after computing.

10.2 Challenge problems

Batch threshold n × n layer: AI = nB ÷ (n + 2B). A100 4096: 223. H100 4096: 345. A100 1024: 330.

Occupancy Blocks = min(registers, threads, block limit, shared memory). Registers bind at 40 × 256 on an A100: 75 percent.

Banks Stride k gives a gcd(k, 32) way conflict. Pad rows to 33 for column reads.

Scan work Hillis and Steele n log n − (n − 1) in log n steps. Blelloch 2(n − 1) in 2 log n steps.

Tile budget Double buffered FP32 with BK = 32 in 48 KB: BM + BN ≤ 192, best 96 × 96, AI 24.

Conflict degree for stride k
degree = gcd(k, 32)

Blelloch adds = 2 (n − 1), steps = 2 $\log_2$ n

10.3 Exam revision

Three formula groups Speed and limits (Amdahl, peak, roofline), threads and memory (index, warps, occupancy, banks, tiling), patterns (reduction, scan, pipeline, formats).

Write the unit Threads or warps, per thread or per block, bytes read and written, FMA = 2 FLOPs, GB = 10^9 bytes.

Slide corrections Thread 343 is (3, 3, 0). Heat centre 0.4. Width³ loads. Sharpen gives 30. Attempt 2 breaks mutual exclusion. C tile in registers.

T4 numbers 40 SMs, 8.14 TFLOP/s FP32, 320 GB/s, ridge 25.4, 1024 threads and 16 blocks per SM.

Next step rule Memory bound: reuse data (tiles). Copy bound: keep data on the GPU or overlap. Latency bound: more warps or more independent work.

Speedup and efficiency
$S = T_{old} \div T_{new}$, $E = S \div N$

Amdahl (fixed problem)
$S(N) = 1 \div ((1 - P) + P \div N)$, ceiling $1 \div (1 - P)$

Peak FLOP/s
peak = SMs × cores per SM × 2 × clock

Arithmetic intensity
AI = FLOPs ÷ bytes moved

Roofline
attainable = min(peak, AI × bandwidth), ridge = peak ÷ bandwidth

Global index and blocks
i = blockIdx.x × blockDim.x + threadIdx.x, blocks = ⌈N ÷ B⌉

Warps
warps per block = ⌈B ÷ 32⌉, lane = t mod 32

Occupancy
blocks per SM = min(threads, blocks, registers, shared memory limits)

Bank conflicts degree = gcd(stride, 32)

Matmul loads naive $2N^3$, tiled $2N^3 \div T$

Reduction tree
$\log_2$ B levels, B − 1 additions, 1 + $\log_2$ B barriers

Scans
Hillis and Steele n $\log_2$ n − (n − 1) adds in $\log_2$ n steps, Blelloch 2(n − 1) adds in 2 $\log_2$ n steps

Pipeline
T = (a + b + c) + (n − 1) × max(a, b, c)

Float value
$x = (-1)^s \times 1.m \times 2^{e - bias}$, $bias = 2^{E-1} - 1$

EXAM Most lost marks come from counting the wrong thing: threads instead of warps, loads per thread instead of per block, bytes one way instead of both. Write the unit next to every number.

EXAM If an exam question copies one of these slides, answer with the right value and add one line saying why. Markers reward the reasoning.